

Architecture Design Report

This document presents the architectural design of the system, including its context, logical structure, runtime behavior, and deployment.

1. System Requirements

System Name: ResuFit: Precision CV Matching System

Architecture Style: Microservices

Functional Requirements

- Submit CV
- View and Edit CV
- Answer Screening Questions
- Get Rating for CV
- Parse CV
- Compare CV with Job Requirements
- Validate and Score Applicant Responses
- Generate Percentage-Based Result

Non-Functional Requirements

- Performance

Description: The system **MUST** respond to user requests within a reasonable time frame.

- Reliability

Description: The system **MUST** be able to recover from failures and maintain data integrity.

- Security

Description: The system **MUST** protect user data and prevent unauthorized access.

- Privacy

Description: The system **MUST** ensure that user data is collected and stored in accordance with applicable laws and regulations.

- Scalability

Description: The system **MUST** be able to handle a large number of users and requests without compromising performance.

- Usability

Description: The system **MUST** provide an intuitive and user-friendly interface that is easy to navigate and use.

- Maintainability

Description: The system MUST be designed and implemented in a way that makes it easy to modify, update,

- Compatibility

Description: The system MUST be compatible with a variety of devices, browsers, and operating systems.

- Integration

Description: The system MUST be able to integrate with other systems and applications as needed.

2. Formal Architecture Specification (ACME ADL)

The following section provides a formal architectural specification using ACME ADL, capturing components, con-

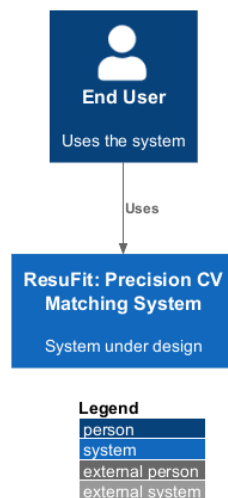
```
System ResuFit:PrecisionCVMatchingSystem {
  Property view : String = "runtime";
  Property architectureStyle : String = "Microservices";
  Property source : String = "FALLBACK";
  Component APIGateway = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Route client requests";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
    Property type : String = "L7";
    Property healthChecks : Boolean = true;
  }
  Component UserService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Manage users";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component OrderService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Manage orders";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component OrderDatabase = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Persist order data";
    Property state : String = "stateful";
    Property scaling : String = "vertical";
    Property consistency : String = "eventual";
    Property replication : String = "multi-node";
  }
  Attachment APIGateway.out to UserService.in {
    Property delivery : String = "at-least-once";
```

```

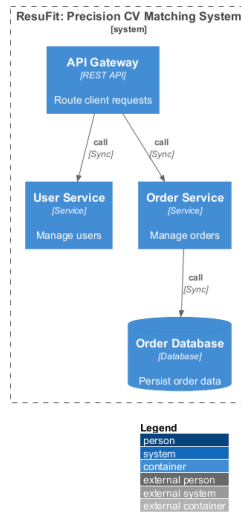
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment APIGateway.out to OrderService.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment OrderService.out to OrderDatabase.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Property resilience_pattern : String = "Retry, CircuitBreaker";
Property delivery : String = "at-least-once";
Property ordering : Boolean = false;
Property authentication : String = "service-to-service";
Property authorization : String = "RBAC";
}

```

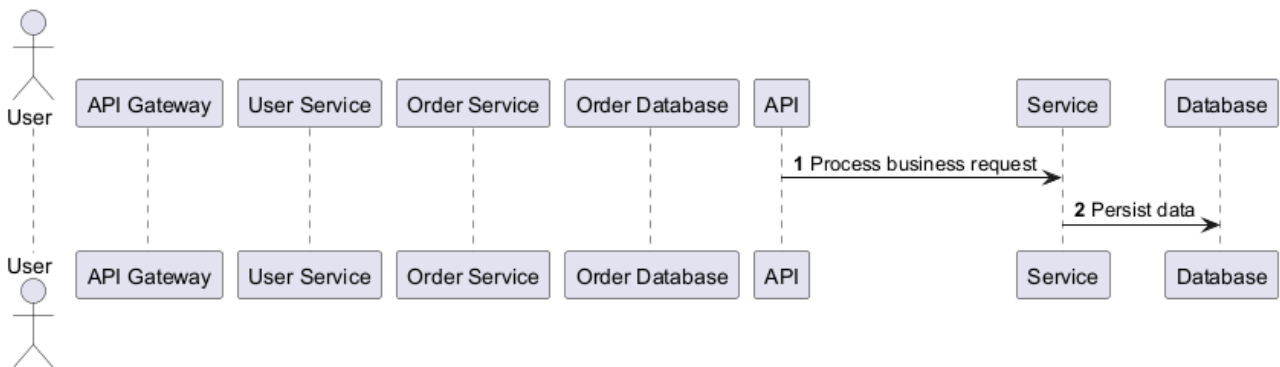
3. Context View



4. Logical View (C4 Container Diagram)



5. Process View (Runtime Interaction)



6. Physical View (Deployment Diagram)

